Railway Management System using Database Applications

by

Divyanth Dayasankar.P , 24BCE1993

A project report submitted

for the course of

BCSE302L - Database Systems

in

B. Tech. COMPUTER SCIENCE AND ENGINEERING

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

April 2026

TABLE OF CONTENTS

Section	Page
Abstract	3
1. Introduction	4
2. Existing Work	5
3. Proposed Work / System	6
4. System Design / Architecture	7
5. Technology Stack	9
6. Working Modules	10
7. Screenshots of Output	12
8. Conclusion	18
9. References	19
10. Complete Implementation	19

ABSTRACT

The Railway Management System is a web-based database application designed to digitize and centralize the management of railway operations. The system manages eight core entities — Stations, Routes, Trains, Engines, Coaches, Depots, Operators, and Passengers — and the relationships between them. The primary objective is to provide an administrative interface through which railway data can be created, read, updated, and deleted in a structured and consistent manner.

The application is built using Django, a Python-based web framework, as the backend, with Oracle Database XE as the relational database management system. The frontend is developed using HTML, CSS, and JavaScript with Django's templating engine. The database schema has been designed following entity-relationship modeling principles and normalized up to Boyce-Codd Normal Form (BCNF) to eliminate redundancy and ensure data integrity. Referential integrity is enforced through foreign key constraints across all related tables.

The system allows administrators to manage train schedules by assigning routes with source and destination stations and departure and arrival times, assign operators to trains, register passengers against specific trains, track engines and coaches linked to trains, and maintain depot records. A central dashboard provides a live count of all records across all eight entities along with the five most recently added trains and passengers.

This project demonstrates the practical application of core database concepts including ER modeling, relational schema design, normalization, SQL DDL and DML operations, and the integration of a relational database with a modern web application framework.

1. INTRODUCTION

Railway systems are among the most data-intensive operations in the transportation sector. Managing thousands of trains, routes, passengers, crew members, and rolling stock demands a reliable, structured, and easily queryable information system. Traditional paper-based or spreadsheet-based approaches are prone to inconsistency, data duplication, and difficulty in retrieving meaningful information quickly.

Database systems offer a systematic solution to these challenges. According to Elmasri and Navathe, a database is "a collection of related data" and a database management system (DBMS) is "a computerized system that enables users to create and maintain a database" [1]. A well-designed relational database, backed by an application layer, allows data to be stored without redundancy, queried efficiently, and maintained with integrity constraints that prevent invalid states.

This project builds a Railway Management System that models the real-world operations of a railway network using a relational database. The system represents eight entities — Station, Route, Train, Engine, Coach, Depot, Operator, and Passenger — and the relationships between them. The relationships range from simple one-to-many (a Train has many Coaches) to more complex ones involving three entities (the "Belongs to" ternary relationship between Coach, Engine, and Train as depicted in the ER diagram).

The application layer, built with Django, abstracts SQL interactions behind Python model objects and provides a web-based interface for all database operations. The backend database is Oracle XE, accessed via the django.db.backends.oracle driver. Every user action on the web interface ultimately translates to a SQL INSERT, UPDATE, DELETE, or SELECT statement executed against the Oracle database.

The scope of this project covers:

- Designing and implementing the relational schema in Oracle
- Normalizing all relations to BCNF
- Building a Django web application for full CRUD operations on all eight entities
- Providing a summary dashboard for administrative overview

2. EXISTING WORK

Railway management has historically been handled through a combination of manual records, legacy mainframe systems, and more recently, enterprise resource planning (ERP) systems. Several approaches exist in both academia and industry.

Manual / Paper-Based Systems: Older railway administrations maintained physical ledgers for passenger bookings, train schedules, and crew assignments. These systems suffer from data duplication, difficulty in cross-referencing records, and no support for concurrent access.

Spreadsheet-Based Systems: Spreadsheet tools like Microsoft Excel have been used to track train schedules and passenger lists. However, spreadsheets lack referential integrity, cannot enforce relationships between data, and become unwieldy at scale.

Centralized RDBMS-Based Systems: Modern railway organizations such as Indian Railways use large-scale RDBMS-backed systems (such as the PRS — Passenger Reservation System) to manage bookings, schedules, and operations. These systems leverage transactions, concurrency control, and constraint enforcement.

Online Booking Portals: Consumer-facing systems like IRCTC expose a subset of the backend database to passengers for ticket booking and cancellation. These are built on top of large relational databases with high-availability configurations.

Research in Database Design for Railways: Academic literature demonstrates the use of ER modeling and relational schema design for transportation domains. Elmasri and Navathe discuss entity-relationship modeling as a tool for designing databases for complex real-world systems [1], covering relationship types (1:1, 1:N, M:N), participation constraints (total and partial), and multi-valued attributes — all of which apply directly to the railway domain.

The limitation of existing systems relevant to this project context is that full-stack implementations exposing all administrative operations in a simplified, understandable form are less commonly demonstrated at the academic level. This project addresses that gap by building a complete administrative system with a transparent database layer.

3. PROPOSED WORK / SYSTEM

The proposed system is a web-based Railway Management System that provides a centralized administrative interface for all railway data. The system is intended for railway administrators who need to manage the full operational data of a railway network.

3.1 Objectives

1. Design a normalized relational schema for a railway management domain with eight entities.
2. Implement the schema in Oracle Database with appropriate primary key and foreign key constraints.
3. Build a Django web application providing full CRUD functionality for all entities.
4. Provide a dashboard summarizing the current state of all eight entities in real time.
5. Enforce referential integrity so that deleting a referenced record does not corrupt related data.

3.2 Scope of the System

The system manages the following data entities:

Entity	Description
Station	Named stops on a rail network, referenced as source/destination by routes.
Operator	Employees assigned to operate trains, with name and contact number.
Route	Named path between source and destination station with departure and arrival times.
Train	Named train assigned to one route and operated by one operator.
Engine	Propulsion unit with type (Diesel/Electric/Steam/Hybrid) and horsepower, linked to a train.
Coach	Passenger carriage with type (AC/Sleeper/General/First Class) and capacity, linked to train and engine.
Depot	Named maintenance and storage facility associated with one coach and one engine.
Passenger	Individual registered onto a specific train with contact information.

3.3 Key Design Decisions

- Foreign keys use ON DELETE SET NULL semantics: deleting a parent record nullifies the foreign key in child records rather than cascading the deletion, preventing accidental data loss.
- All primary keys are auto-incremented integers managed by Oracle sequences and triggers.
- The schema is in BCNF, ensuring no redundancy or update anomalies.
- Django's select_related() is used for all list views to avoid N+1 query problems by issuing a single SQL JOIN.

4. SYSTEM DESIGN / ARCHITECTURE

4.1 ER Diagram

The system contains 8 entities modeled with the following relationships:

Relationship	Entities Involved	Type
Operates	Operator → Train	1:N
Forms / Follows	Train → Route → Station	N:1 (Train to Route), Route references Station twice
Ticket In	Passenger → Train	N:1
Belongs to	Coach ↔ Train ↔ Engine	Ternary (Coach linked to both Train and Engine)
Stores	Depot → Coach, Depot → Engine	1:1 FK references

(Refer to the ER diagram submitted in DA1)

4.2 Relational Schema

```
Station   (Station_ID [PK], Name)
Operator  (Employee_ID [PK], Name, Phone_Number)
Route     (Route_ID [PK], Name, SStation_ID [FK→Station], STime,
           DStation_ID [FK→Station], DTime)
Train     (Train_ID [PK], Name, Employee_ID [FK→Operator], Route_ID [FK→Route])
Engine    (Engine_ID [PK], Horsepower, Type, Train_ID [FK→Train])
Coach     (Coach_ID [PK], Capacity, Type, Engine_ID [FK→Engine], Train_ID
[FK→Train])
Depot     (Depot_ID [PK], Name, Coach_ID [FK→Coach], Engine_ID [FK→Engine])
Passenger (Unique_ID [PK], Name, Phone_Number, Train_ID [FK→Train])
```

4.3 Normalization

All eight relations satisfy 1NF, 2NF, 3NF, and BCNF. A formal proof for each relation was presented in DA2. The key conclusions are:

Normal Form	Condition	Status for All 8 Relations
1NF	All attributes are atomic; no multivalued or composite attributes.	Satisfied — all attributes are simple and single-valued.
2NF	No partial dependency on the primary key.	Satisfied — all primary keys are single-attribute, so partial dependency is impossible.
3NF	No transitive dependency.	Satisfied — all non-key attributes depend directly on the primary key only.
BCNF	For every non-trivial FD X→A, X must be a superkey.	Satisfied — the only FD in each relation has the primary key as the determinant, which is a superkey.

4.4 Application Architecture — MVT Pattern

The system follows Django's MVT (Model-View-Template) architecture:

```
Browser (HTTP Request)
        |
        v
    urls.py  —— Routes URL to the correct view function
        |
        v
    views.py —— Business logic; queries the database via ORM
        |
        v
   models.py —— Python classes mapped to Oracle tables via
django.db.backends.oracle
        |
        v
  Oracle XE DB —— Executes SQL; returns result sets
        |
        v
    views.py —— Passes context dictionary to template
        |
        v
  templates/ —— Django templates render HTML with context data
        |
        v
Browser (HTTP Response)
```

5. TECHNOLOGY STACK

Component	Technology	Details
Backend Framework	Django 6.0.2	Python web framework, MVT architecture
Programming Language	Python 3.x	Application logic, ORM
Database	Oracle Database XE	Relational DBMS at localhost:1521/XEPDB1
DB Schema	RAILWAY user	All 8 tables under the RAILWAY schema
ORM	Django ORM	Maps Python model classes to Oracle tables
Frontend	HTML5, CSS3, JavaScript	UI rendered via Django templates
Template Engine	Django Templates	Server-side HTML rendering with context data
Web Server	Django Dev Server	manage.py runserver on port 8000
Oracle Driver	django.db.backends.oracle	Python-Oracle connectivity via cx_Oracle
Auto-Increment	Oracle Sequences + Triggers	One sequence and trigger per table for PK generation

Database Connection Configuration

```
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.oracle',
        'NAME': 'localhost:1521/XEPDB1',
        'USER': 'railway',
        'PASSWORD': 'railway123',
    }
}
```

6. WORKING MODULES

The application consists of 9 functional modules: one Dashboard and eight CRUD modules, one per entity. All CRUD modules follow an identical pattern: List → Add / Edit / Delete. The system exposes 32 URL endpoints in total (4 per entity × 8 entities).

6.1 Dashboard Module

URL: /

The dashboard is the home page of the application. It queries all eight tables using Django ORM count() calls and passes the totals to the template as context variables. It also fetches the 5 most recently added trains and passengers using select_related() to perform efficient JOIN queries and avoid N+1 database calls.

Context Variable	Query	Displayed As
train_count	Train.objects.count()	Stat card
route_count	Route.objects.count()	Stat card
station_count	Station.objects.count()	Stat card
passenger_count	Passenger.objects.count()	Stat card
operator_count	Operator.objects.count()	Stat card
coach_count	Coach.objects.count()	Stat card
engine_count	Engine.objects.count()	Stat card
depot_count	Depot.objects.count()	Stat card
recent_trains	Train.objects.select_related().all()[:5]	Table — last 5 trains
recent_passengers	Passenger.objects.select_related().all()[:5]	Table — last 5 passengers

6.2 Station Module

URLs: /stations/ /stations/add/ /stations/<pk>/edit/ /stations/<pk>/delete/

Model: Station(station_id [PK], name). The list view fetches all stations ordered by station_id and renders them in a table with columns ID and Name. The add and edit views use StationForm (single field: name). The delete view shows a confirmation page before permanently removing the record.

6.3 Operator Module

URLs: /operators/ /operators/add/ /operators/<pk>/edit/ /operators/<pk>/delete/

Model: Operator(employee_id [PK], name, phone_number). The list view displays ID, Name, and Phone Number. Phone number is optional (nullable). The form exposes name and phone_number fields. Full CRUD operations are supported.

6.4 Route Module

URLs: /routes/ /routes/add/ /routes/<pk>/edit/ /routes/<pk>/delete/

Model: Route(route_id [PK], name, sstation [FK→Station], stime, dstation [FK→Station], dtime). The Route entity references the Station table twice — once for the source station (sstation) and once for the destination station (dstation) — using different related_name values (departure_routes and arrival_routes). The list view displays ID, Name, From (source station name), Departure Time, To (destination station name), and Arrival Time. The form uses HTML <input type="time"> for the time fields and a select dropdown for station choices. Uses select_related('sstation', 'dstation') for efficient querying.

6.5 Train Module

URLs: /trains/ /trains/add/ /trains/<pk>/edit/ /trains/<pk>/delete/

Model: Train(train_id [PK], name, employee [FK→Operator], route [FK→Route]). A train is assigned to exactly one operator and follows exactly one route. The list view displays ID, Name, Operator, and Route using select_related('employee', 'route'). The form exposes name, employee (dropdown from Operator table), and route (dropdown from Route table). Full CRUD operations are supported.

6.6 Engine Module

URLs: /engines/ /engines/add/ /engines/<pk>/edit/ /engines/<pk>/delete/

Model: Engine(engine_id [PK], horsepower, type [choices: Diesel/Electric/Steam/Hybrid], train [FK→Train]). Each engine is associated with one train. The type field is constrained to four valid choices enforced at the application layer. The list view displays ID, Type, Horsepower, and Train using select_related('train'). Full CRUD operations are supported.

6.7 Coach Module

URLs: /coaches/ /coaches/add/ /coaches/<pk>/edit/ /coaches/<pk>/delete/

Model: Coach(coach_id [PK], capacity, type [choices: AC/Sleeper/General/First Class], engine [FK→Engine], train [FK→Train]). The Coach entity is linked to both a Train and an Engine, reflecting the ternary "Belongs to" relationship from the ER diagram. The list view displays ID, Type, Capacity, Engine, and Train using select_related('engine', 'train'). Full CRUD operations are supported.

6.8 Depot Module

URLs: /depots/ /depots/add/ /depots/<pk>/edit/ /depots/<pk>/delete/

Model: Depot(depot_id [PK], name, coach [FK→Coach, SET_NULL], engine [FK→Engine, SET_NULL]). A depot is a named maintenance and storage facility that holds one coach and one engine. Both foreign keys use SET_NULL on delete, meaning if a referenced coach or engine is deleted, the depot record is preserved with NULL values rather than being deleted. The list view displays ID, Name, Coach, and Engine using select_related('coach', 'engine'). Full CRUD operations are supported.

6.9 Passenger Module

URLs: /passengers/ /passengers/add/ /passengers/<pk>/edit/ /passengers/<pk>/delete/

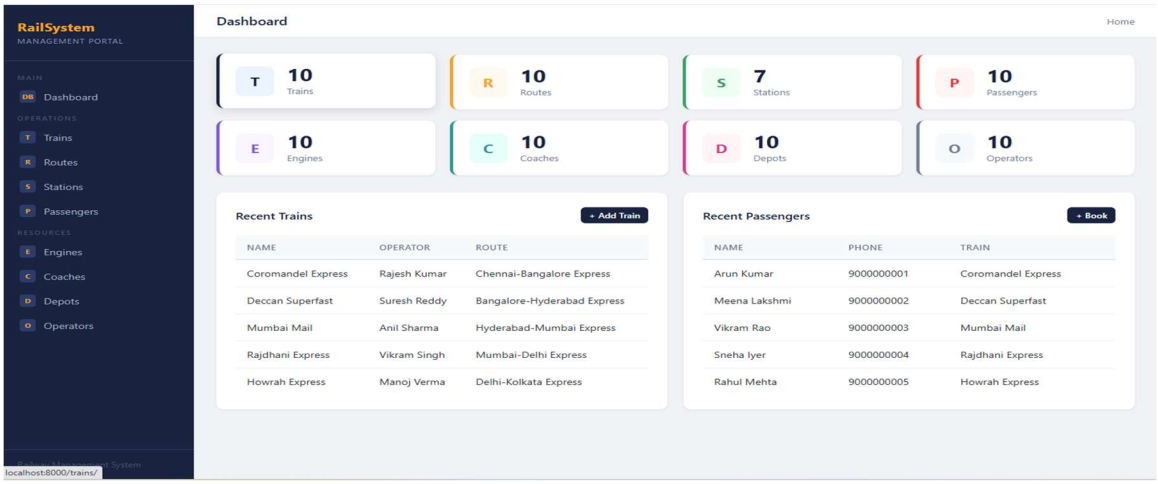
Model: Passenger(unique_id [PK], name, phone_number [nullable], train [FK→Train]). Passengers are registered against a specific train. The add operation is labeled "Book Passenger" to reflect the railway domain context. Phone number is optional. The list view displays ID, Name, Phone, and Train using select_related('train'). Full CRUD operations are supported.

7. SCREENSHOTS OF OUTPUT

All data shown is the sample data inserted via the SQL INSERT statements documented in DA2.

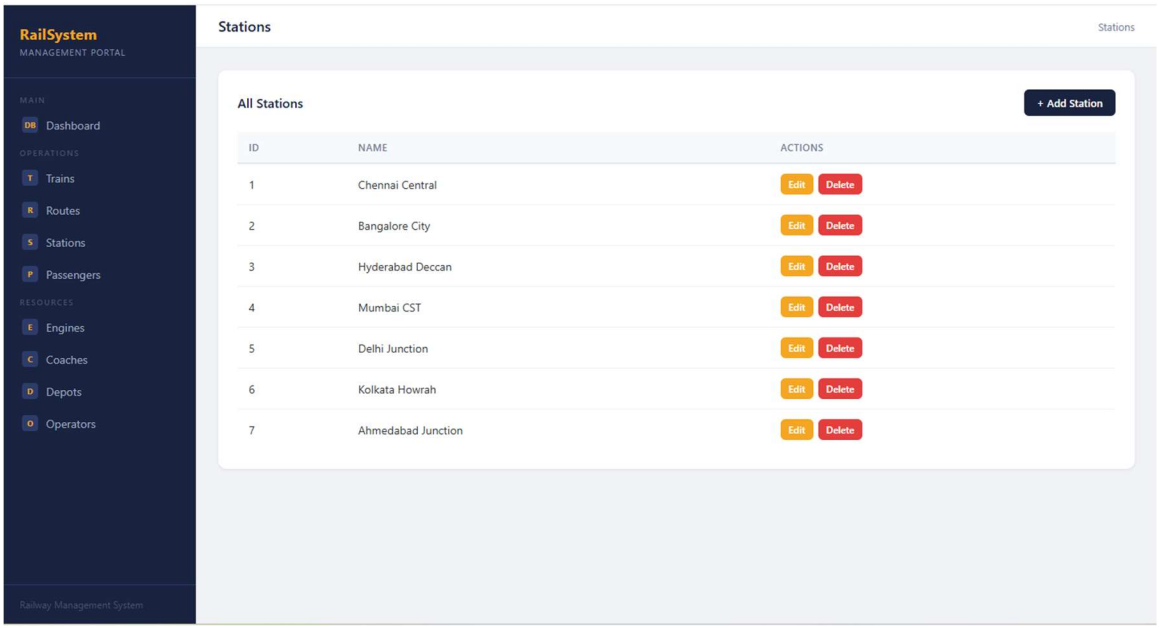
7.1 Dashboard

Stat cards for all 8 entities + recent trains and passengers tables.



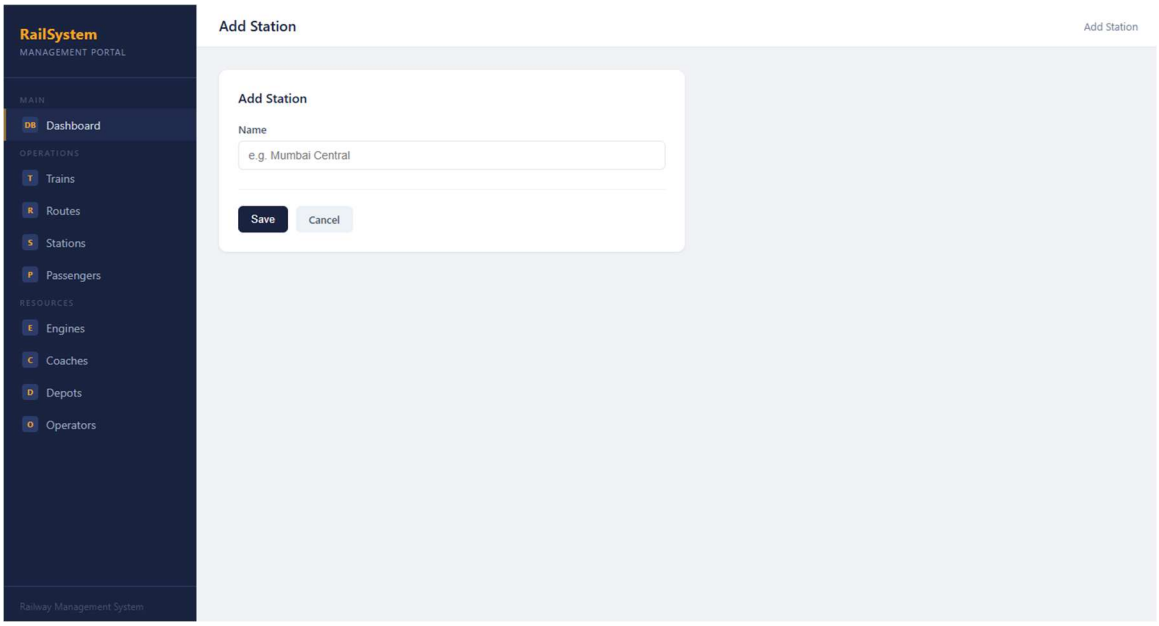
7.2 Station List

Table listing all stations with ID and Name columns.



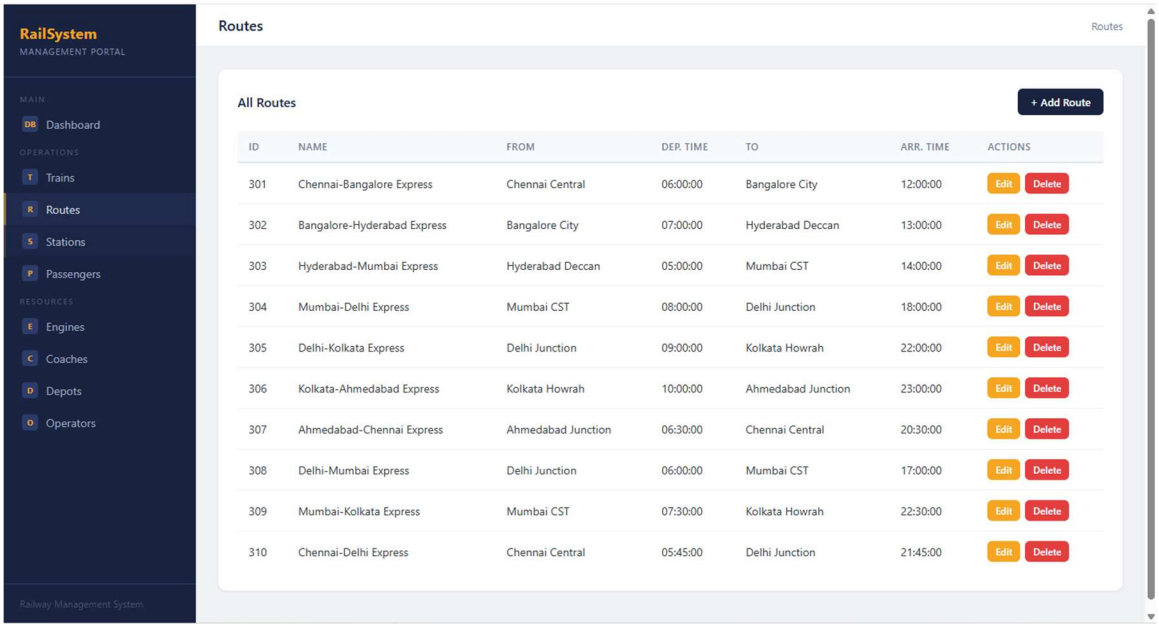
7.3 Add Station Form

Form with a single Name input field.



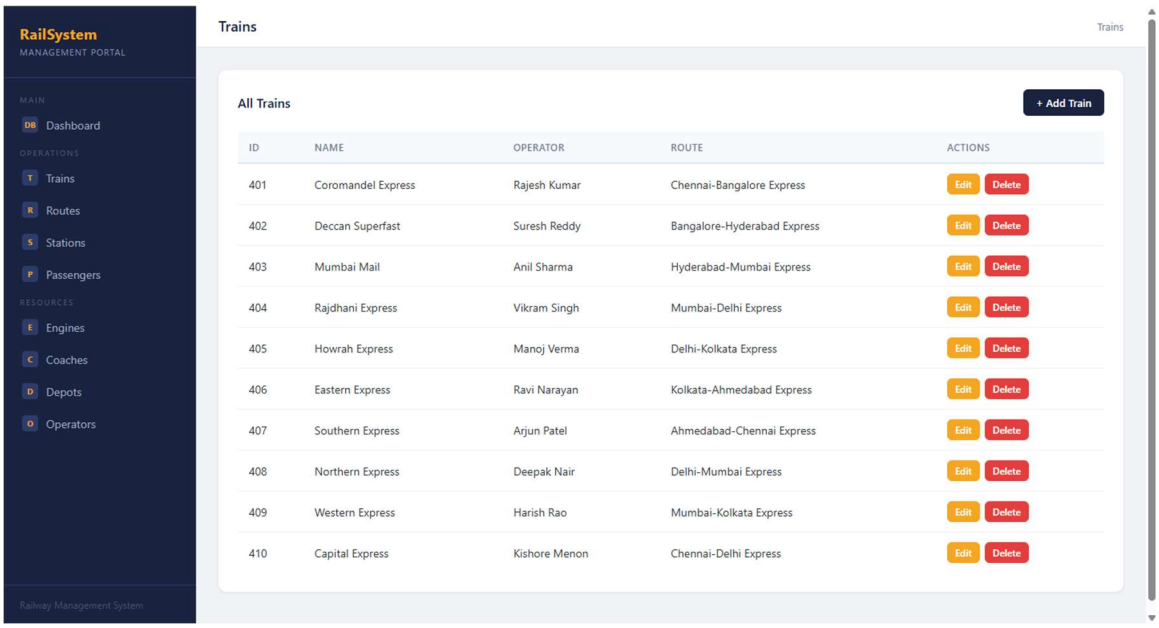
7.4 Route List

Table with columns: ID, Name, From, Departure Time, To, Arrival Time.



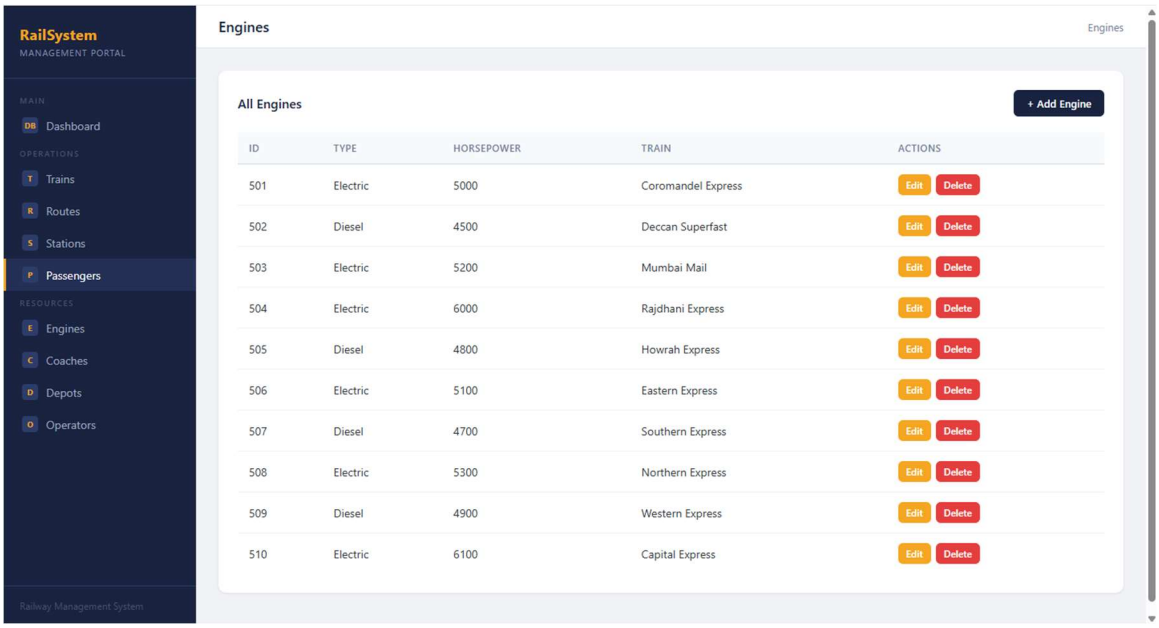
7.5 Train List

Table showing Train ID, Name, assigned Operator, and Route.



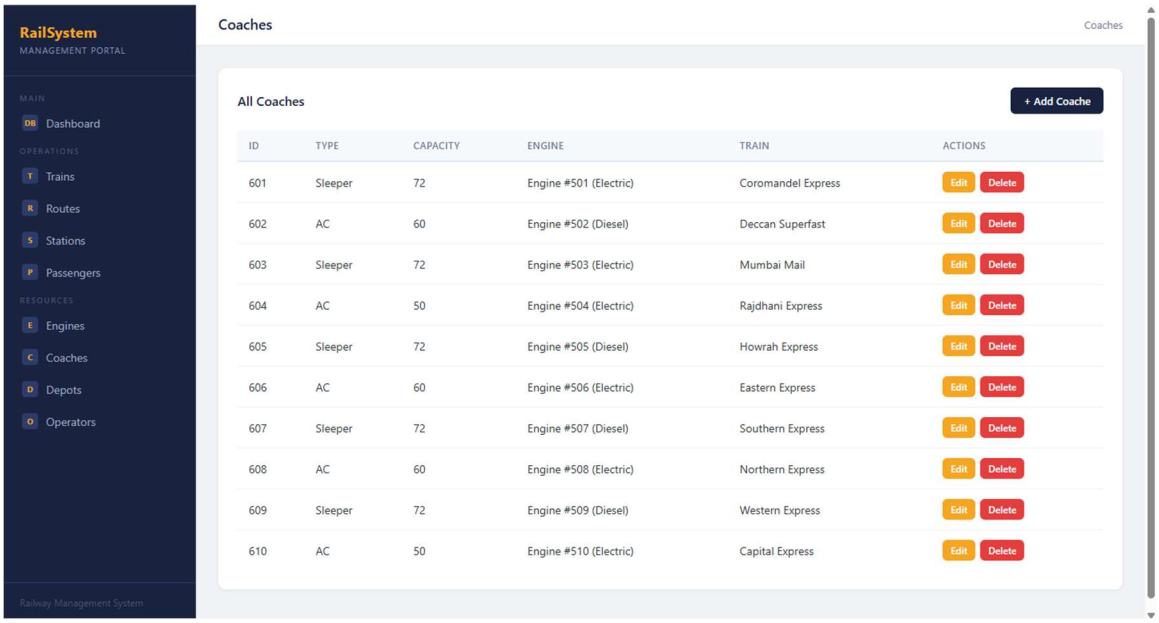
7.6 Engine List

Table showing Engine ID, Type, Horsepower, and assigned Train.



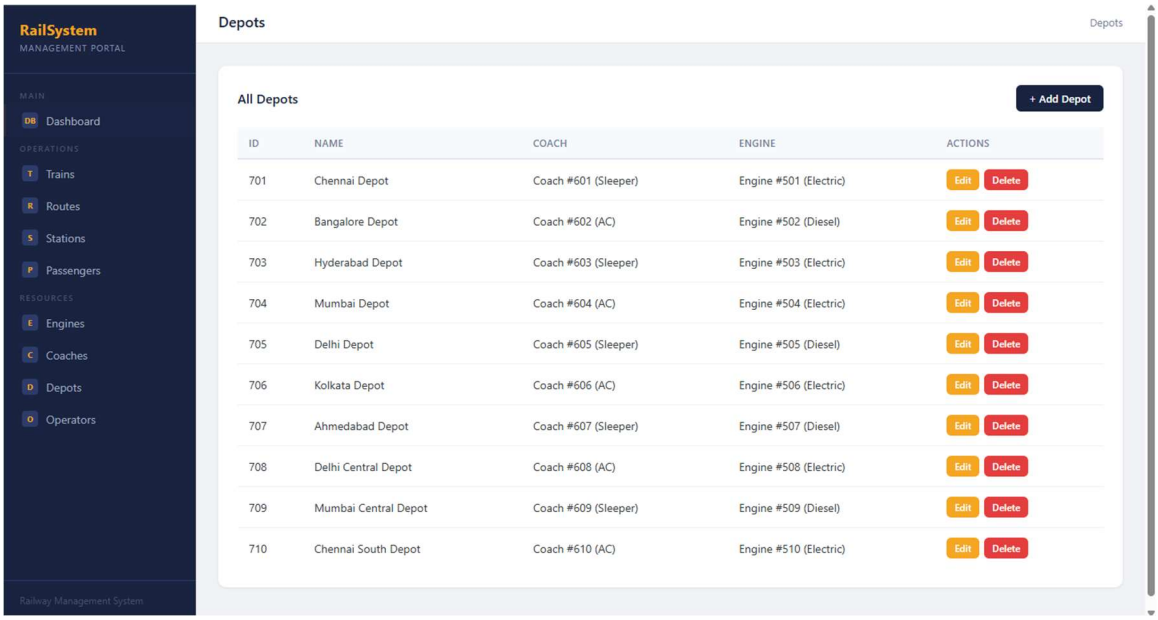
7.7 Coach List

Table showing Coach ID, Type, Capacity, Engine, and Train.



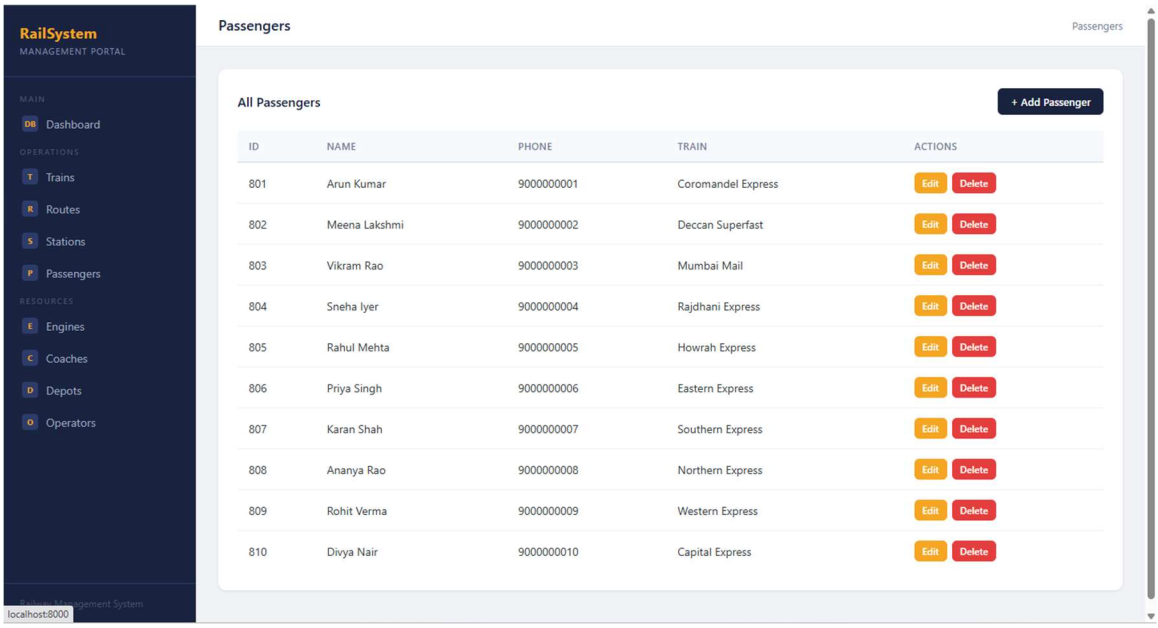
7.8 Depot List

Table showing Depot ID, Name, linked Coach, and Engine.



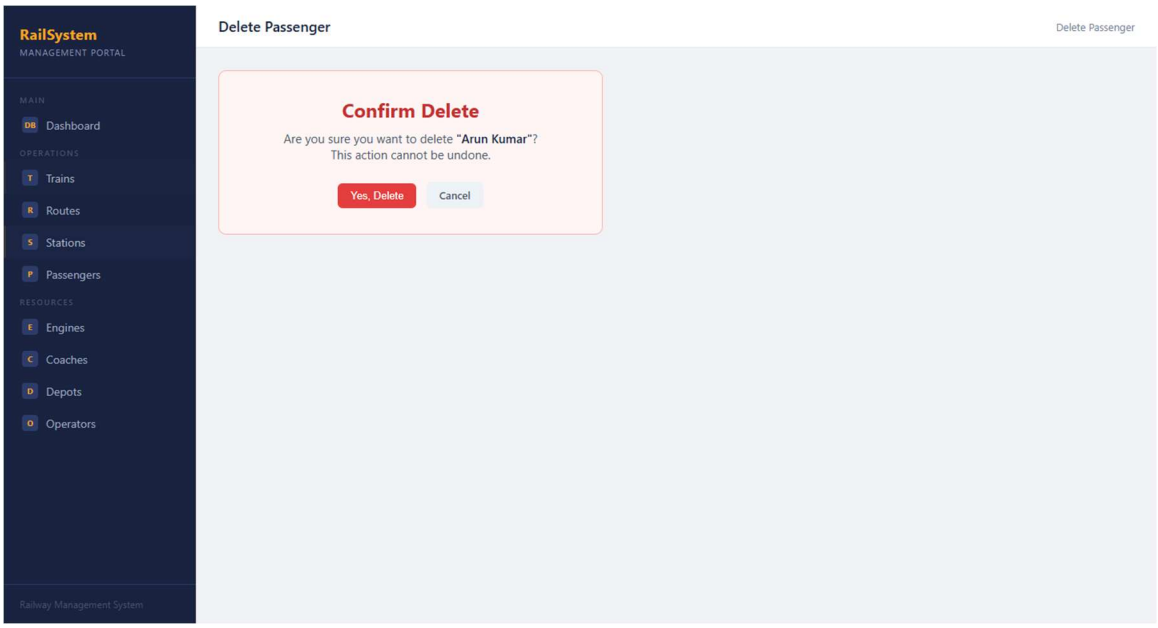
7.9 Passenger List

Table showing Passenger ID, Name, Phone Number, and Train.



7.10 Delete Confirmation

Confirmation dialog asking to confirm deletion of a record.



8. CONCLUSION

This project successfully demonstrates the design and implementation of a Railway Management System using relational database principles and a Django web application framework. The following outcomes were achieved:

ER Modeling: Eight entities and their relationships — including 1:1, 1:N, N:M, and a ternary relationship — were identified and modeled using the ER diagram methodology described in Elmasri and Navathe [1].

Relational Schema Design: The ER diagram was mapped to eight relational tables with foreign key relationships established between all dependent entities.

Normalization: All eight relations were formally verified to satisfy 1NF, 2NF, 3NF, and BCNF. Since every relation has a single-attribute primary key with no transitive dependencies, normalization was achieved naturally and all relations are in BCNF.

Oracle Implementation: All tables were created in Oracle XE using DDL statements with appropriate primary key and foreign key constraints. Sample data was populated using DML INSERT statements for all eight tables with 7–10 rows each.

Django Web Application: A complete CRUD web application was developed exposing all 32 URL endpoints (4 per entity × 8 entities). The application uses Django's ORM to interface with Oracle, Django forms for input validation, and Django templates for rendering the user interface.

Data Integrity: ON DELETE SET NULL semantics on all foreign keys ensure that deleting a parent record does not cascade and destroy child records, maintaining the integrity of the dataset.

The system demonstrates that principles taught in database theory — ER modeling, schema mapping, normalization, and SQL — directly apply to building practical, real-world web applications. Future enhancements could include user authentication, advanced search and filtering, ticket booking with seat management, and reporting dashboards with aggregate queries.

9. REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ: Pearson, 2016.

[2] Django Software Foundation, Django Documentation, Version 6.0. Available: https://docs.djangoproject.com/

[3] Oracle Corporation, Oracle Database Express Edition Documentation. Available: https://docs.oracle.com/

[4] W3Schools, SQL Tutorial. Available: https://www.w3schools.com/sql/

10. COMPLETE IMPLEMENTATION

10.1 models.py

```
from django.db import models


class Station(models.Model):
    station_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)

    class Meta:
        db_table = 'station'

    def __str__(self):
        return self.name


class Operator(models.Model):
    employee_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    phone_number = models.CharField(max_length=15, blank=True, null=True)

    class Meta:
        db_table = 'operator'

    def __str__(self):
        return self.name


class Route(models.Model):
    route_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    sstation = models.ForeignKey(
        Station, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='departure_routes', verbose_name='Source Station'
```

```
    )
    stime = models.TimeField(blank=True, null=True, verbose_name='Departure
Time')
    dstation = models.ForeignKey(
        Station, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='arrival_routes', verbose_name='Destination Station'
    )
    dtime = models.TimeField(blank=True, null=True, verbose_name='Arrival
Time')

    class Meta:
        db_table = 'route'

    def __str__(self):
        return self.name



class Train(models.Model):
    train_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    employee = models.ForeignKey(
        Operator, on_delete=models.SET_NULL, null=True, blank=True,
        verbose_name='Operator'
    )
    route = models.ForeignKey(
        Route, on_delete=models.SET_NULL, null=True, blank=True
    )

    class Meta:
        db_table = 'train'

    def __str__(self):
        return self.name



class Engine(models.Model):
    ENGINE_TYPES = [
        ('Diesel', 'Diesel'), ('Electric', 'Electric'),
        ('Steam', 'Steam'),   ('Hybrid', 'Hybrid'),
    ]
    engine_id  = models.AutoField(primary_key=True)
    horsepower = models.IntegerField(blank=True, null=True)
    type       = models.CharField(max_length=50, blank=True, null=True,
                                  choices=ENGINE_TYPES)
    train      = models.ForeignKey(
        Train, on_delete=models.SET_NULL, null=True, blank=True
    )

    class Meta:
        db_table = 'engine'

    def __str__(self):
```

```
        return f"Engine #{self.engine_id} ({self.type})"


class Coach(models.Model):
    COACH_TYPES = [
        ('AC', 'AC'), ('Sleeper', 'Sleeper'),
        ('General', 'General'), ('First Class', 'First Class'),
    ]
    coach_id = models.AutoField(primary_key=True)
    capacity = models.IntegerField(blank=True, null=True)
    type     = models.CharField(max_length=50, blank=True, null=True,
                                choices=COACH_TYPES)
    engine   = models.ForeignKey(
        Engine, on_delete=models.SET_NULL, null=True, blank=True
    )
    train    = models.ForeignKey(
        Train, on_delete=models.SET_NULL, null=True, blank=True
    )

    class Meta:
        db_table = 'coach'

    def __str__(self):
        return f"Coach #{self.coach_id} ({self.type})"


class Depot(models.Model):
    depot_id = models.AutoField(primary_key=True)
    name     = models.CharField(max_length=100)
    coach    = models.ForeignKey(
        Coach, on_delete=models.SET_NULL, null=True, blank=True
    )
    engine   = models.ForeignKey(
        Engine, on_delete=models.SET_NULL, null=True, blank=True
    )

    class Meta:
        db_table = 'depot'

    def __str__(self):
        return self.name


class Passenger(models.Model):
    unique_id    = models.AutoField(primary_key=True)
    name         = models.CharField(max_length=100)
    phone_number = models.CharField(max_length=15, blank=True, null=True)
    train        = models.ForeignKey(
        Train, on_delete=models.SET_NULL, null=True, blank=True
    )

    class Meta:
```

```
        db_table = 'passenger'

    def __str__(self):
        return self.name
```

10.2 forms.py

```
from django import forms
from .models import Station, Operator, Route, Train, Engine, Coach, Depot,
Passenger


class StationForm(forms.ModelForm):
    class Meta:
        model  = Station
        fields = ['name']
        widgets = {
            'name': forms.TextInput(attrs={'placeholder': 'e.g. Mumbai
Central'}),
        }


class OperatorForm(forms.ModelForm):
    class Meta:
        model  = Operator
        fields = ['name', 'phone_number']


class RouteForm(forms.ModelForm):
    class Meta:
        model   = Route
        fields  = ['name', 'sstation', 'stime', 'dstation', 'dtime']
        widgets = {
            'stime': forms.TimeInput(attrs={'type': 'time'}),
            'dtime': forms.TimeInput(attrs={'type': 'time'}),
        }


class TrainForm(forms.ModelForm):
    class Meta:
        model  = Train
        fields = ['name', 'employee', 'route']


class EngineForm(forms.ModelForm):
    class Meta:
        model  = Engine
        fields = ['horsepower', 'type', 'train']
```

```
class CoachForm(forms.ModelForm):
    class Meta:
        model  = Coach
        fields = ['capacity', 'type', 'engine', 'train']


class DepotForm(forms.ModelForm):
    class Meta:
        model  = Depot
        fields = ['name', 'coach', 'engine']


class PassengerForm(forms.ModelForm):
    class Meta:
        model  = Passenger
        fields = ['name', 'phone_number', 'train']
```

10.3 views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages
from .models import Station, Operator, Route, Train, Engine, Coach, Depot,
Passenger
from .forms import (
    StationForm, OperatorForm, RouteForm, TrainForm,
    EngineForm, CoachForm, DepotForm, PassengerForm
)


# ── DASHBOARD ──────────────────────────────────────────────────────────

def dashboard(request):
    return render(request, 'railway/dashboard.html', {
        'train_count':        Train.objects.count(),
        'route_count':        Route.objects.count(),
        'station_count':      Station.objects.count(),
        'passenger_count':    Passenger.objects.count(),
        'operator_count':     Operator.objects.count(),
        'coach_count':        Coach.objects.count(),
        'engine_count':       Engine.objects.count(),
        'depot_count':        Depot.objects.count(),
        'recent_trains':      Train.objects.select_related('route',
'employee').all()[:5],
        'recent_passengers':
Passenger.objects.select_related('train').all()[:5],
    })


# ── STATIONS ───────────────────────────────────────────────────────────

def station_list(request):
```

```
    stations = Station.objects.all().order_by('station_id')
    rows = [{'pk': s.pk, 'cells': [s.station_id, s.name]} for s in stations]
    return render(request, 'railway/list.html', {
        'title': 'Stations', 'icon': 'S', 'headers': ['ID', 'Name'],
        'rows': rows, 'add_url_name': 'station_add',
        'edit_url_name': 'station_edit', 'delete_url_name': 'station_delete',
    })

def station_add(request):
    form = StationForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Station added successfully.')
        return redirect('station_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Station', 'form': form, 'back_url_name': 'station_list',
    })

def station_edit(request, pk):
    station = get_object_or_404(Station, pk=pk)
    form = StationForm(request.POST or None, instance=station)
    if form.is_valid():
        form.save()
        messages.success(request, 'Station updated successfully.')
        return redirect('station_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Station', 'form': form, 'back_url_name': 'station_list',
    })

def station_delete(request, pk):
    station = get_object_or_404(Station, pk=pk)
    if request.method == 'POST':
        station.delete()
        messages.success(request, 'Station deleted.')
        return redirect('station_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Station', 'object_name': str(station),
        'back_url_name': 'station_list',
    })


# ── OPERATORS ──────────────────────────────────────────────────────────

def operator_list(request):
    operators = Operator.objects.all().order_by('employee_id')
    rows = [{'pk': o.pk, 'cells': [o.employee_id, o.name, o.phone_number or '-
']}
            for o in operators]
    return render(request, 'railway/list.html', {
        'title': 'Operators', 'icon': 'O', 'headers': ['ID', 'Name', 'Phone'],
        'rows': rows, 'add_url_name': 'operator_add',
        'edit_url_name': 'operator_edit', 'delete_url_name': 'operator_delete',
    })
```

```
def operator_add(request):
    form = OperatorForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Operator added successfully.')
        return redirect('operator_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Operator', 'form': form, 'back_url_name':
'operator_list',
    })

def operator_edit(request, pk):
    operator = get_object_or_404(Operator, pk=pk)
    form = OperatorForm(request.POST or None, instance=operator)
    if form.is_valid():
        form.save()
        messages.success(request, 'Operator updated successfully.')
        return redirect('operator_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Operator', 'form': form, 'back_url_name':
'operator_list',
    })

def operator_delete(request, pk):
    operator = get_object_or_404(Operator, pk=pk)
    if request.method == 'POST':
        operator.delete()
        messages.success(request, 'Operator deleted.')
        return redirect('operator_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Operator', 'object_name': str(operator),
        'back_url_name': 'operator_list',
    })


# ── ROUTES ──────────────────────────────────────────────────────────

def route_list(request):
    routes = Route.objects.select_related('sstation',
'dstation').all().order_by('route_id')
    rows = [
        {'pk': r.pk, 'cells': [
            r.route_id, r.name,
            str(r.sstation) if r.sstation else '-',
            str(r.stime)    if r.stime    else '-',
            str(r.dstation) if r.dstation else '-',
            str(r.dtime)    if r.dtime    else '-',
        ]}
        for r in routes
    ]
    return render(request, 'railway/list.html', {
        'title': 'Routes', 'icon': 'R',
```

```
        'headers': ['ID', 'Name', 'From', 'Dep. Time', 'To', 'Arr. Time'],
        'rows': rows, 'add_url_name': 'route_add',
        'edit_url_name': 'route_edit', 'delete_url_name': 'route_delete',
    })

def route_add(request):
    form = RouteForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Route added successfully.')
        return redirect('route_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Route', 'form': form, 'back_url_name': 'route_list',
    })

def route_edit(request, pk):
    route = get_object_or_404(Route, pk=pk)
    form = RouteForm(request.POST or None, instance=route)
    if form.is_valid():
        form.save()
        messages.success(request, 'Route updated successfully.')
        return redirect('route_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Route', 'form': form, 'back_url_name': 'route_list',
    })

def route_delete(request, pk):
    route = get_object_or_404(Route, pk=pk)
    if request.method == 'POST':
        route.delete()
        messages.success(request, 'Route deleted.')
        return redirect('route_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Route', 'object_name': str(route),
        'back_url_name': 'route_list',
    })


# ── TRAINS ──────────────────────────────────────────────────────────

def train_list(request):
    trains = Train.objects.select_related('employee',
'route').all().order_by('train_id')
    rows = [
        {'pk': t.pk, 'cells': [
            t.train_id, t.name,
            str(t.employee) if t.employee else '-',
            str(t.route)    if t.route    else '-',
        ]}
        for t in trains
    ]
    return render(request, 'railway/list.html', {
        'title': 'Trains', 'icon': 'T',
```

```
        'headers': ['ID', 'Name', 'Operator', 'Route'],
        'rows': rows, 'add_url_name': 'train_add',
        'edit_url_name': 'train_edit', 'delete_url_name': 'train_delete',
    })

def train_add(request):
    form = TrainForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Train added successfully.')
        return redirect('train_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Train', 'form': form, 'back_url_name': 'train_list',
    })

def train_edit(request, pk):
    train = get_object_or_404(Train, pk=pk)
    form = TrainForm(request.POST or None, instance=train)
    if form.is_valid():
        form.save()
        messages.success(request, 'Train updated successfully.')
        return redirect('train_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Train', 'form': form, 'back_url_name': 'train_list',
    })

def train_delete(request, pk):
    train = get_object_or_404(Train, pk=pk)
    if request.method == 'POST':
        train.delete()
        messages.success(request, 'Train deleted.')
        return redirect('train_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Train', 'object_name': str(train),
        'back_url_name': 'train_list',
    })


# ── ENGINES ──────────────────────────────────────────────────────────

def engine_list(request):
    engines =
Engine.objects.select_related('train').all().order_by('engine_id')
    rows = [
        {'pk': e.pk, 'cells': [
            e.engine_id, e.type or '-', e.horsepower or '-',
            str(e.train) if e.train else '-',
        ]}
        for e in engines
    ]
    return render(request, 'railway/list.html', {
        'title': 'Engines', 'icon': 'E',
        'headers': ['ID', 'Type', 'Horsepower', 'Train'],
```

```
        'rows': rows, 'add_url_name': 'engine_add',
        'edit_url_name': 'engine_edit', 'delete_url_name': 'engine_delete',
    })

def engine_add(request):
    form = EngineForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Engine added successfully.')
        return redirect('engine_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Engine', 'form': form, 'back_url_name': 'engine_list',
    })

def engine_edit(request, pk):
    engine = get_object_or_404(Engine, pk=pk)
    form = EngineForm(request.POST or None, instance=engine)
    if form.is_valid():
        form.save()
        messages.success(request, 'Engine updated successfully.')
        return redirect('engine_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Engine', 'form': form, 'back_url_name': 'engine_list',
    })

def engine_delete(request, pk):
    engine = get_object_or_404(Engine, pk=pk)
    if request.method == 'POST':
        engine.delete()
        messages.success(request, 'Engine deleted.')
        return redirect('engine_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Engine', 'object_name': str(engine),
        'back_url_name': 'engine_list',
    })


# ── COACHES ──────────────────────────────────────────────────────────

def coach_list(request):
    coaches = Coach.objects.select_related('engine',
'train').all().order_by('coach_id')
    rows = [
        {'pk': c.pk, 'cells': [
            c.coach_id, c.type or '-', c.capacity or '-',
            str(c.engine) if c.engine else '-',
            str(c.train)  if c.train  else '-',
        ]}
        for c in coaches
    ]
    return render(request, 'railway/list.html', {
        'title': 'Coaches', 'icon': 'C',
        'headers': ['ID', 'Type', 'Capacity', 'Engine', 'Train'],
```

```
        'rows': rows, 'add_url_name': 'coach_add',
        'edit_url_name': 'coach_edit', 'delete_url_name': 'coach_delete',
    })

def coach_add(request):
    form = CoachForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Coach added successfully.')
        return redirect('coach_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Coach', 'form': form, 'back_url_name': 'coach_list',
    })

def coach_edit(request, pk):
    coach = get_object_or_404(Coach, pk=pk)
    form = CoachForm(request.POST or None, instance=coach)
    if form.is_valid():
        form.save()
        messages.success(request, 'Coach updated successfully.')
        return redirect('coach_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Coach', 'form': form, 'back_url_name': 'coach_list',
    })

def coach_delete(request, pk):
    coach = get_object_or_404(Coach, pk=pk)
    if request.method == 'POST':
        coach.delete()
        messages.success(request, 'Coach deleted.')
        return redirect('coach_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Coach', 'object_name': str(coach),
        'back_url_name': 'coach_list',
    })


# ── DEPOTS ──────────────────────────────────────────────────────────

def depot_list(request):
    depots = Depot.objects.select_related('coach',
'engine').all().order_by('depot_id')
    rows = [
        {'pk': d.pk, 'cells': [
            d.depot_id, d.name,
            str(d.coach)  if d.coach  else '-',
            str(d.engine) if d.engine else '-',
        ]}
        for d in depots
    ]
    return render(request, 'railway/list.html', {
        'title': 'Depots', 'icon': 'D',
        'headers': ['ID', 'Name', 'Coach', 'Engine'],
```

```
        'rows': rows, 'add_url_name': 'depot_add',
        'edit_url_name': 'depot_edit', 'delete_url_name': 'depot_delete',
    })

def depot_add(request):
    form = DepotForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Depot added successfully.')
        return redirect('depot_list')
    return render(request, 'railway/form.html', {
        'title': 'Add Depot', 'form': form, 'back_url_name': 'depot_list',
    })

def depot_edit(request, pk):
    depot = get_object_or_404(Depot, pk=pk)
    form = DepotForm(request.POST or None, instance=depot)
    if form.is_valid():
        form.save()
        messages.success(request, 'Depot updated successfully.')
        return redirect('depot_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Depot', 'form': form, 'back_url_name': 'depot_list',
    })

def depot_delete(request, pk):
    depot = get_object_or_404(Depot, pk=pk)
    if request.method == 'POST':
        depot.delete()
        messages.success(request, 'Depot deleted.')
        return redirect('depot_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Depot', 'object_name': str(depot),
        'back_url_name': 'depot_list',
    })


# ── PASSENGERS ──────────────────────────────────────────────────────

def passenger_list(request):
    passengers =
Passenger.objects.select_related('train').all().order_by('unique_id')
    rows = [
        {'pk': p.pk, 'cells': [
            p.unique_id, p.name, p.phone_number or '-',
            str(p.train) if p.train else '-',
        ]}
        for p in passengers
    ]
    return render(request, 'railway/list.html', {
        'title': 'Passengers', 'icon': 'P',
        'headers': ['ID', 'Name', 'Phone', 'Train'],
        'rows': rows, 'add_url_name': 'passenger_add',
```

```
        'edit_url_name': 'passenger_edit', 'delete_url_name':
'passenger_delete',
    })

def passenger_add(request):
    form = PassengerForm(request.POST or None)
    if form.is_valid():
        form.save()
        messages.success(request, 'Passenger booked successfully.')
        return redirect('passenger_list')
    return render(request, 'railway/form.html', {
        'title': 'Book Passenger', 'form': form, 'back_url_name':
'passenger_list',
    })

def passenger_edit(request, pk):
    passenger = get_object_or_404(Passenger, pk=pk)
    form = PassengerForm(request.POST or None, instance=passenger)
    if form.is_valid():
        form.save()
        messages.success(request, 'Passenger updated successfully.')
        return redirect('passenger_list')
    return render(request, 'railway/form.html', {
        'title': 'Edit Passenger', 'form': form, 'back_url_name':
'passenger_list',
    })

def passenger_delete(request, pk):
    passenger = get_object_or_404(Passenger, pk=pk)
    if request.method == 'POST':
        passenger.delete()
        messages.success(request, 'Passenger removed.')
        return redirect('passenger_list')
    return render(request, 'railway/confirm_delete.html', {
        'title': 'Delete Passenger', 'object_name': str(passenger),
        'back_url_name': 'passenger_list',
    })
```

10.4 urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.dashboard, name='dashboard'),

    # Stations
    path('stations/',                  views.station_list,
name='station_list'),
    path('stations/add/',              views.station_add,
name='station_add'),
```

```
    path('stations/<int:pk>/edit/',    views.station_edit,
name='station_edit'),
    path('stations/<int:pk>/delete/',  views.station_delete,
name='station_delete'),

    # Operators
    path('operators/',                 views.operator_list,
name='operator_list'),
    path('operators/add/',             views.operator_add,
name='operator_add'),
    path('operators/<int:pk>/edit/',   views.operator_edit,
name='operator_edit'),
    path('operators/<int:pk>/delete/', views.operator_delete,
name='operator_delete'),

    # Routes
    path('routes/',                    views.route_list,   name='route_list'),
    path('routes/add/',                views.route_add,    name='route_add'),
    path('routes/<int:pk>/edit/',      views.route_edit,   name='route_edit'),
    path('routes/<int:pk>/delete/',    views.route_delete,
name='route_delete'),

    # Trains
    path('trains/',                    views.train_list,   name='train_list'),
    path('trains/add/',                views.train_add,    name='train_add'),
    path('trains/<int:pk>/edit/',      views.train_edit,   name='train_edit'),
    path('trains/<int:pk>/delete/',    views.train_delete,
name='train_delete'),

    # Engines
    path('engines/',                   views.engine_list,
name='engine_list'),
    path('engines/add/',               views.engine_add,    name='engine_add'),
    path('engines/<int:pk>/edit/',     views.engine_edit,
name='engine_edit'),
    path('engines/<int:pk>/delete/',   views.engine_delete,
name='engine_delete'),

    # Coaches
    path('coaches/',                   views.coach_list,   name='coach_list'),
    path('coaches/add/',               views.coach_add,    name='coach_add'),
    path('coaches/<int:pk>/edit/',     views.coach_edit,   name='coach_edit'),
    path('coaches/<int:pk>/delete/',   views.coach_delete,
name='coach_delete'),

    # Depots
    path('depots/',                    views.depot_list,   name='depot_list'),
    path('depots/add/',                views.depot_add,    name='depot_add'),
    path('depots/<int:pk>/edit/',      views.depot_edit,   name='depot_edit'),
    path('depots/<int:pk>/delete/',    views.depot_delete,
name='depot_delete'),

    # Passengers
```

```
    path('passengers/',                 views.passenger_list,
name='passenger_list'),
    path('passengers/add/',             views.passenger_add,
name='passenger_add'),
    path('passengers/<int:pk>/edit/',  views.passenger_edit,
name='passenger_edit'),
    path('passengers/<int:pk>/delete/',views.passenger_delete,
name='passenger_delete'),
]
```

10.5 Oracle SQL — Table Creation (DDL)

```
CREATE TABLE Station (
    Station_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL
);

CREATE TABLE Operator (
    Employee_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    Phone_Number VARCHAR2(15)
);

CREATE TABLE Route (
    Route_ID    NUMBER PRIMARY KEY,
    Name        VARCHAR2(100) NOT NULL,
    SStation_ID NUMBER,
    STime       DATE,
    DStation_ID NUMBER,
    DTime       DATE,
    CONSTRAINT fk_route_source
        FOREIGN KEY (SStation_ID) REFERENCES Station(Station_ID),
    CONSTRAINT fk_route_destination
        FOREIGN KEY (DStation_ID) REFERENCES Station(Station_ID)
);

CREATE TABLE Train (
    Train_ID    NUMBER PRIMARY KEY,
    Name        VARCHAR2(100) NOT NULL,
    Employee_ID NUMBER,
    Route_ID    NUMBER,
    CONSTRAINT fk_train_operator
        FOREIGN KEY (Employee_ID) REFERENCES Operator(Employee_ID),
    CONSTRAINT fk_train_route
        FOREIGN KEY (Route_ID) REFERENCES Route(Route_ID)
);

CREATE TABLE Engine (
    Engine_ID  NUMBER PRIMARY KEY,
    Horsepower NUMBER,
    Type       VARCHAR2(50),
    Train_ID   NUMBER,
```

```
    CONSTRAINT fk_engine_train
        FOREIGN KEY (Train_ID) REFERENCES Train(Train_ID)
);

CREATE TABLE Coach (
    Coach_ID  NUMBER PRIMARY KEY,
    Capacity  NUMBER,
    Type      VARCHAR2(50),
    Engine_ID NUMBER,
    Train_ID  NUMBER,
    CONSTRAINT fk_coach_engine
        FOREIGN KEY (Engine_ID) REFERENCES Engine(Engine_ID),
    CONSTRAINT fk_coach_train
        FOREIGN KEY (Train_ID) REFERENCES Train(Train_ID)
);

CREATE TABLE Depot (
    Depot_ID  NUMBER PRIMARY KEY,
    Name      VARCHAR2(100) NOT NULL,
    Coach_ID  NUMBER,
    Engine_ID NUMBER,
    CONSTRAINT fk_depot_coach
        FOREIGN KEY (Coach_ID) REFERENCES Coach(Coach_ID),
    CONSTRAINT fk_depot_engine
        FOREIGN KEY (Engine_ID) REFERENCES Engine(Engine_ID)
);

CREATE TABLE Passenger (
    Unique_ID    NUMBER PRIMARY KEY,
    Name         VARCHAR2(100) NOT NULL,
    Phone_Number VARCHAR2(15),
    Train_ID     NUMBER,
    CONSTRAINT fk_passenger_train
        FOREIGN KEY (Train_ID) REFERENCES Train(Train_ID)
);
```

10.6 Oracle SQL — Sequences and Triggers (Auto-Increment)

```
-- Station
CREATE SEQUENCE station_sq  START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER station_tr
BEFORE INSERT ON station FOR EACH ROW WHEN (new.station_id IS NULL)
BEGIN SELECT station_sq.NEXTVAL INTO :new.station_id FROM DUAL; END;
/

-- Operator
CREATE SEQUENCE operator_sq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER operator_tr
BEFORE INSERT ON operator FOR EACH ROW WHEN (new.employee_id IS NULL)
BEGIN SELECT operator_sq.NEXTVAL INTO :new.employee_id FROM DUAL; END;
/
```

```
-- Route
CREATE SEQUENCE route_sq    START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER route_tr
BEFORE INSERT ON route FOR EACH ROW WHEN (new.route_id IS NULL)
BEGIN SELECT route_sq.NEXTVAL INTO :new.route_id FROM DUAL; END;
/

-- Train
CREATE SEQUENCE train_sq    START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER train_tr
BEFORE INSERT ON train FOR EACH ROW WHEN (new.train_id IS NULL)
BEGIN SELECT train_sq.NEXTVAL INTO :new.train_id FROM DUAL; END;
/

-- Engine
CREATE SEQUENCE engine_sq   START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER engine_tr
BEFORE INSERT ON engine FOR EACH ROW WHEN (new.engine_id IS NULL)
BEGIN SELECT engine_sq.NEXTVAL INTO :new.engine_id FROM DUAL; END;
/

-- Coach
CREATE SEQUENCE coach_sq    START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER coach_tr
BEFORE INSERT ON coach FOR EACH ROW WHEN (new.coach_id IS NULL)
BEGIN SELECT coach_sq.NEXTVAL INTO :new.coach_id FROM DUAL; END;
/

-- Depot
CREATE SEQUENCE depot_sq    START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER depot_tr
BEFORE INSERT ON depot FOR EACH ROW WHEN (new.depot_id IS NULL)
BEGIN SELECT depot_sq.NEXTVAL INTO :new.depot_id FROM DUAL; END;
/

-- Passenger
CREATE SEQUENCE passenger_sq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE OR REPLACE TRIGGER passenger_tr
BEFORE INSERT ON passenger FOR EACH ROW WHEN (new.unique_id IS NULL)
BEGIN SELECT passenger_sq.NEXTVAL INTO :new.unique_id FROM DUAL; END;
/
```